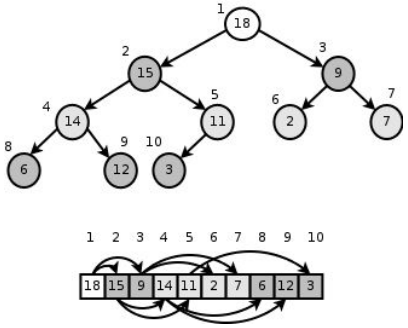
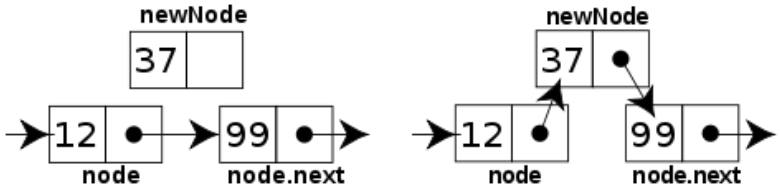



```
INSERTION-SORT(A)                          cost    times
for j = 2 to A.length                      c1      n
    key = A[j]                             c2      n − 1
    // Insert A[j] into the sorted
        sequence A[1 .. j − 1].            0       n − 1
    i = j − 1                              c4      n − 1
    while i > 0 and A[i] > key             c5      Σ_{j=2}^{n} t_j
        A[i + 1] = A[i]                    c6      Σ_{j=2}^{n} (t_j − 1)
        i = i − 1                          c7      Σ_{j=2}^{n} (t_j − 1)
    A[i + 1] = key                         c8      n − 1
```



WELCOME TO CS 24!

Problem Solving with Computers-II

Instructor: Diba Mirza

```
#include <iostream>
using namespace std;

int main(){
    cout<<"Hola Facebook\n";
    return 0;
}
```

Read the syllabus. Know what's required. Know how to get help.

Course website: https://ucsb-cs24.github.io/s26

What this course is really about

Small vocabulary. Graphs, trees, hash tables, heaps, tries…

Infinite problems. Google search. GPS. AI.
Compression. Databases.

One skill. Given a new algorithmic problem,
choose the right approach/data structure,
build a solution,
analyze and defend your choices.

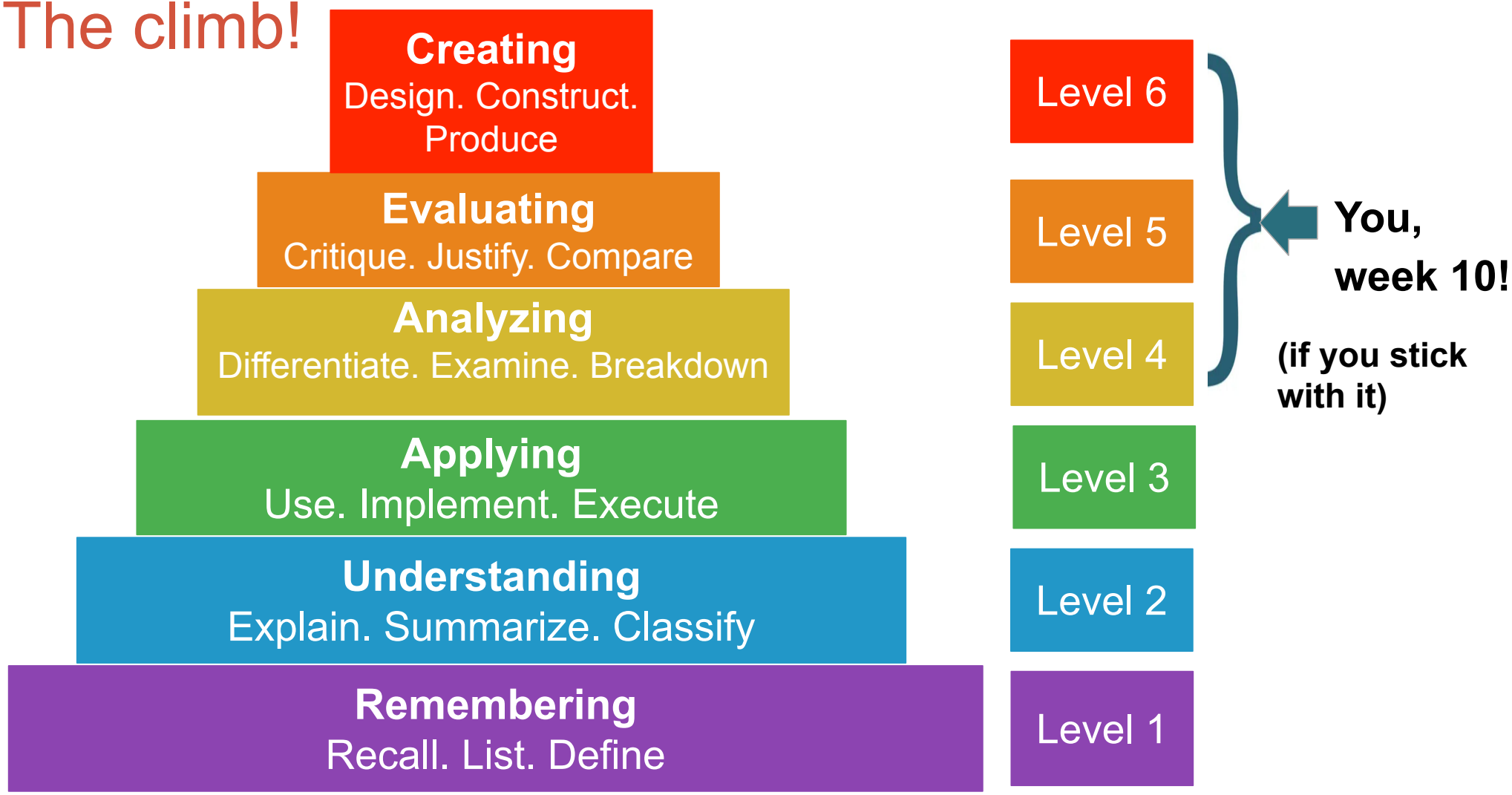
That's what this course trains. Explicitly. Repeatedly.

Goals for CS24

By the end of this course you will be able to:

- Implement and use core data structures from scratch
- Analyze the time and space complexity of your solutions
- Choose the right data structure for a given problem and justify why
- Solve an unseen algorithmic problem and articulate your reasoning
- Evaluate and critique competing solutions

The climb!


Creating
Design. Construct. Produce
Evaluating
Critique. Justify. Compare
Analyzing
Differentiate. Examine. Breakdown
Applying
Use. Implement. Execute
Understanding
Explain. Summarize. Classify
Remembering
Recall. List. Define
Level 6
Level 5
Level 4
Level 3
Level 2
Level 1
You, week 10!
(if you stick with it)

How you will be assessed

Quizzes (4)	**20%**	Concepts + one interview-style problem pen and paper, during lecture
Programming Assignments	**25%**	Build and implement labs 10% | PAs 15%

Interview Prep

LeetCode + Reflection	**10%**	Solve problems, then articulate why your solution works and how you'd improve it

(LeetCode = industry's go-to platform for interview prep)

Mock Interview	**10%**	Unseen problem, think out loud in front of someone → The skill that gets you hired

Final Exam	**35%**	In person, Jun 9, noon–3pm (replaces lowest quiz score if higher)

Cheering you along will be….

- CS24 TAs and LAs and Professor Mirza
- Office hours on Google Calendar posted on the class website

TAs: Sarah Guanyu

LAs: Nikhil Olivia Christian

On AI use

Bottom line: Use AI as a tutor, not a ghostwriter.

Why it matters here specifically: 65% of your grade is evaluated without AI, in real time, in front of a person. The exams will reflect your actual understanding.

What that means in practice:

- ✓ Ask AI to explain a concept you don't understand
- ✓ Write your algorithm first, then use AI to refine
- ✓ Study AI solutions to learn patterns. Then close it and write your own
- ✗ Paste AI code you can't explain line by line
- ✗ Use AI to avoid the struggle. *The struggle is the learning!!*

LeetCode problems: do these without AI. Treat them as exam practice.

Course Logistics

Website: ucsb-cs24.github.io/s26
schedule, assignments, setup — Everything is there.

Syllabus: Read it. Details live there.

Meet your neighbors

Turn to the person sitting near you.

Introduce yourself:

- Your name
- Where you're at in CS
- One thing you're hoping to get out of this class

You have 60 seconds.

How this class works

Phones away unless using them for class.

Lectures are problem-solving sessions, not textbook recap.

Expect to: discuss with neighbors, trace through code,
answer questions, work through problems live.

You learn by doing.

LeetCode Problem

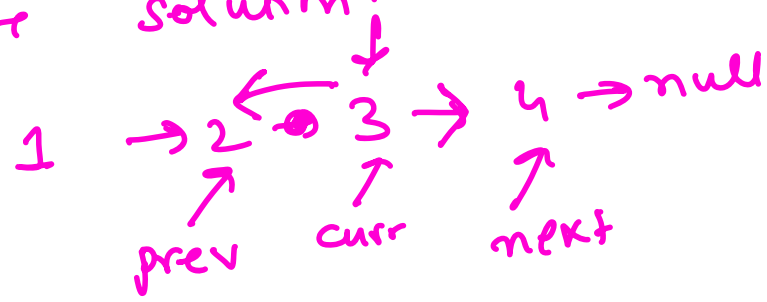
Given the head of a singly linked list, reverse the list, and return the reversed list.

Step 1: Demonstrate that you understand the problem

Input: head → 1 → 2 → 3 → null

Output: head → 3 → 2 → 1 → null.

Step 2: Iterative solution:



Initialization
[prev = nullptr
curr = head

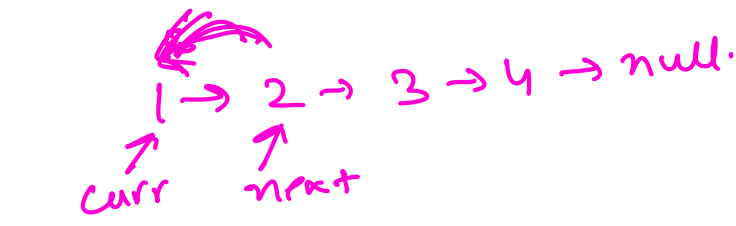
Each iteration:
→ next = curr→next;
→ curr→next = prev
→ Progress forward by one node

return the new head

Link to problem on LeetCode:
https://leetcode.com/problems/reverse-linked-list/description/?envType=problem-list-v2&envId=linked-list

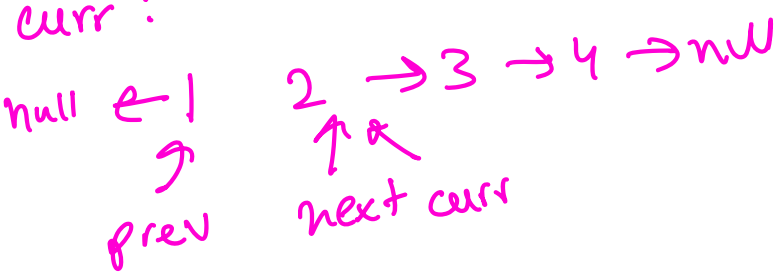
Trace our rough plan (pseudo code) on an example

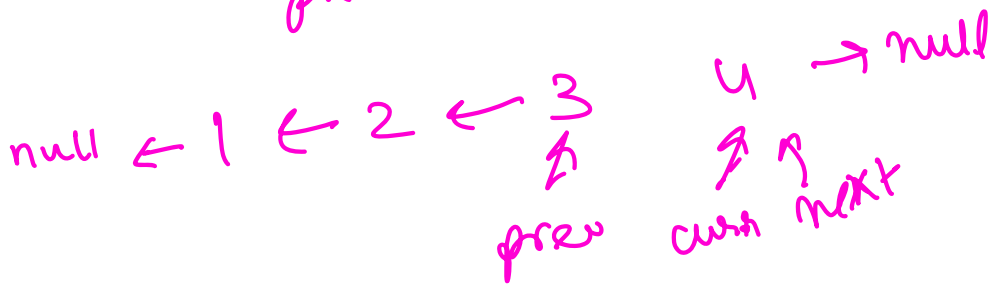
```
// Initialize prev, curr, next
while ( curr ) {
    next = curr->next
    curr->next = prev
    prev = curr;
    curr = next.
}
return prev.
```

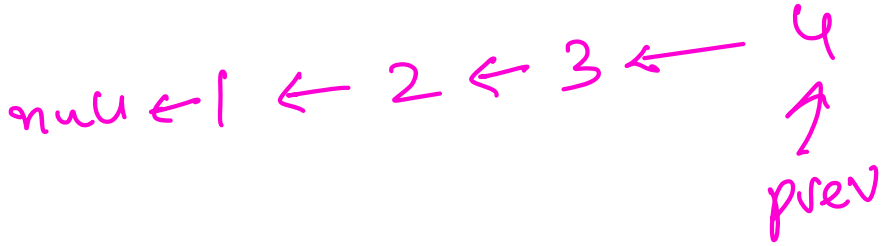


prev : nullptr

curr :







curr = null

next = null

Check edge cases: Does the proposed solution work on an empty list? Yes!

Does it work on a list with one node? Yes!

What would a recursive approach look like?

head → 1 → 2 → 3 → 4 → null

Define the recursive function:

ListNode * reverseList (ListNode * head);

Input: head of a linked list

Returns: head of the reversed list.

Use the definition of the function to construct the recursive algo

---

Base case(s):

(1) if empty list return nullptr

(2) if single node list return head

! (head → next)

Recursive case

Safe to assume input linked list has 2 nodes or more (base cases handle empty & one-node list)

call reverseList to reverse the sublist

1 → [2 → 3 → 4 → null]

head

⇓

[4 → 3 → 2 → null] reversed sublist

new-head

head → 1

The first node still points to node 2.

Rewire 1 & 2 to get the desired outcome

4 → 3 → 2 → 1 → null

Rewire logic

head→next→next = head;

head→next = null;

We wrote the final code, discussed merits of each approach

Think through any problem

1. Restate: show expected output for your own chosen example (pick one different from what you're given)
2. Clarify: constraints, edge cases, assumptions
3. Approach: commit and explain why
   - Iterative: what is the work done each iteration?
   - Recursive: what is the base case? what does one call do?
4. Trace the general case by hand before coding
5. Check edge cases. Does the general solution handle them?
6. Code cleanly. Choose meaningful names, readable structure
7. Test with a different example
8. Complexity — time, space, and tradeoffs between approaches

The steps aren't the point. Your ability to think through a new problem is most important.

One skill to rule them all

Small vocabulary. Graphs, trees, hash tables, heaps, tries…

Infinite problems. Google search. GPS. AI.
Compression. Databases.

One skill. Given a new algorithmic problem,
choose the right approach/data structure,
build a solution,
analyze and defend your choices.

That's what this course trains. Explicitly. Repeatedly.

Next time

- Abstract Data Types and Operator Overloading
- Be sure to do the required reading listed on the course website